Quiz — 1.2.2 Applications Generation — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (8 marks — 1 each) 1. **C** — a defragmenter maintains/optimises the computer itself → utility software. 2. **B** — open source means source code is available and modifiable (usually free, community support). 3. **B** — a compiler translates all at once before running and produces a standalone executable. 4. **B** — an assembler converts assembly mnemonics to machine code, roughly one-to-one. 5. **B** — syntax errors are detected during syntax analysis (parsing) against the grammar. 6. **B** — lexical analysis turns source into tokens, removing whitespace/comments and building a symbol table. 7. **B** — the loader copies the executable from storage into RAM and prepares it to run. 8. **B** — dynamic links are separate files linked at run time, smaller, shareable and updatable.

Section B (12 marks)

9. **[2]** Application software lets the user perform a real-world task; utility software maintains/optimises the computer itself (1). Example pair, e.g. word processor / browser (application) and antivirus / defrag / backup (utility) (1).

10. **[2]** Correct order: **1. Lexical analysis → 2. Syntax analysis → 3. Code generation → 4. Code optimisation.** (2 if all four correct; 1 if one pair transposed.)

11. **[2]** Any two (1 each): translates line by line so it **stops at the first error**, making errors easier to locate; no need to recompile the whole program after each change (faster edit-test cycle); code is portable/can run on any machine with the interpreter; good for testing partial programs.

12. **[3]** A library is a collection of **pre-written, pre-compiled and tested** subroutines (1). Reasons (1 each, max 2): saves development time / avoids rewriting code; more reliable because the code is already tested; promotes reuse/consistency.

13. **[3]** A linker combines the compiled program with library code and other modules into a single executable, resolving references between them (1). **Static** linking copies the library code into the executable (larger, self-contained) (1); **dynamic** linking keeps the library separate (e.g. .dll) and links it at run time (smaller, shared, updatable) (1).

Section C (5 marks) 14. **[5]** Up to 5 from: a compiler translates the whole program into machine code **before** distribution, producing a standalone **executable** (1); only the .exe is shipped, so customers receive machine code, not source — protecting the **source code** from being seen or modified (1); the executable runs **without** needing the compiler/interpreter or source present (1), so customers need no extra software (1); it also runs **faster** as it is already machine code (1). (An interpreter would require the source plus the interpreter at run time, exposing the code — hence unsuitable.)

Total: 8 + 12 + 5 = 25